

MI BF API

Version 1.2

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
1.0	● Initial release	09/12/2020
1.1	● Modified API	10/15/2020
1.2	● 4 MIC Beamforming and modified API	08/03/2021

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. 概述	1
1.1. 算法说明	1
1.2. 关键词说明	1
1.2.1. 噪音阈值	1
1.2.2. 对角加载	1
1.3. 注意	1
1.3.1. 使用	1
1.3.2. 搭配	1
1.3.3. 区分	1
2. 规格	2
3. 数组系统及坐标系	3
3.1. 多麦克风数组	3
3.1.1. 均匀线性数组及坐标系	3
3.1.2. 均匀圆形数组及坐标系	3
4. API 参考	5
4.1. 功能模块 API	5
4.2. IaaBf_GetBufferSize	5
4.3. IaaBf_Init	6
4.4. IaaBf_SetConfig	7
4.5. IaaBf_GetConfig	8
4.6. IaaBf_SetShape	8
4.7. IaaBf_Run	9
4.8. IaaBf_Reset	17
4.9. IaaBf_Free	18
5. BF 数据类型	19
5.1. BF 模块相关数据类型定义	19
5.2. AudioBfInit	19
5.3. AudioBfConfig	20

5.4. BF_HANDLE.....21

6. 错误码.....23

7. 流程图.....24

1. 概述

1.1. 算法说明

BF (beamforming) 波束形成或空间滤波是一种用于传感器数组的定向信号处理技术传输或接收。通过这样一种方式组合麦克风数组中的组件来实现特定角度的信号加强，而其他的则会衰减。波束形成可用于发射端和接收端，以实现空间选择性。

1.2. 关键词说明

1.2.1. 噪音阈值

Noise Gate (dBFS), 低于此值将会把该帧作为噪声部分处理。若设置太高，容易造成语音失真。若设置太低，会影响算法效果。应小心处理此值设定。此参数必须考虑实际麦克风所录制的音量大小，若麦克风录制音量太小且此值设定较大，则该帧会当成噪声部分处理，此值范围为 $[-80,0]$ ，建议值为 **-20**，步长为 1。

1.2.2. 对角加载

透过在矩阵之对角线元素增加对角加载(Diagonal loading)，可以保证反矩阵必定存在达成正则化，因而提升 BF 滤波器的强健性。应小心处理此值设定，若设置太高，会影响算法效果，若设置太低，容易造成语音失真，此值范围为 $[100,1]$ ，建议值为 **10**，步长为 1。

1.3. 注意

1.3.1. 使用

为方便调试和确认算法效果，需要用户应用自行实现替换算法参数和抓取音频数据的逻辑。

1.3.2. 搭配

此模块搭配音频处理链(Audio Process Chain) 会有最佳的信号加强、压制干扰的效果。

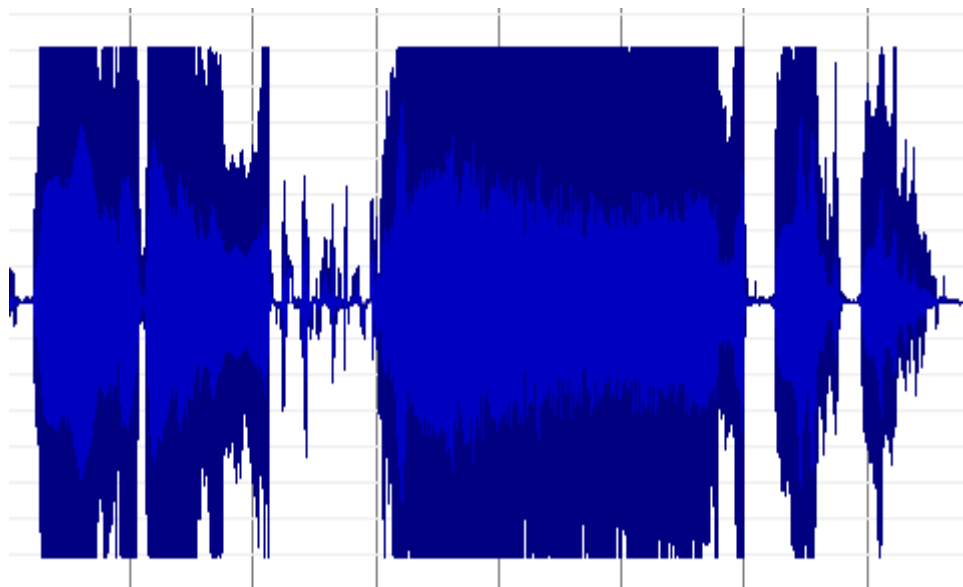
1.3.3. 区分

不同通道数会对应到不同参考库。用户需要自行确认是否使用正确的通道数及对应之参考库。

2. 规格

1. 麦克风个数必须多于两颗。
2. 必须使用同规格的麦克风。
3. 麦克风增益必须相同。
4. 在安静的背景环境，可以达到最好的效果。
5. 使用音档文件做作为声源时，必须确保

A. 音文件中没有发生讯号剪裁, 如图一所示。



图一 讯号剪裁示意图

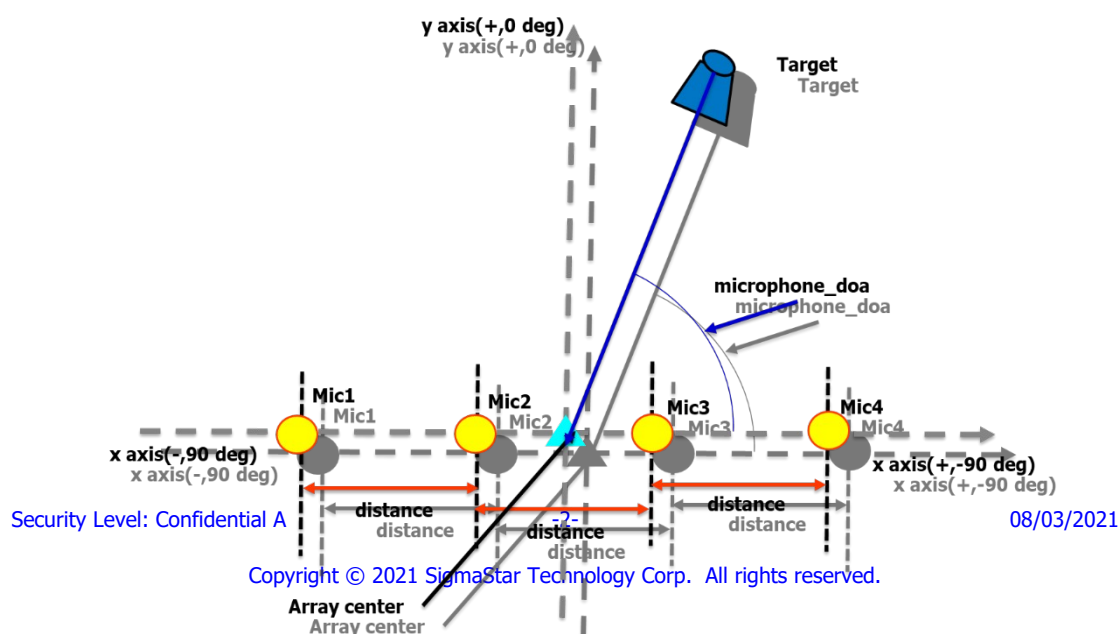
3. 数组系统及坐标系

3.1. 多麦克风数组

多麦克风数组相较于双麦克风数组, 拥有更好的带宽分辨率以及更多组麦克风权重, 可以提升定位以及空间滤波的表现而达到更好的特定角度加强效果, 其中特定角度即为声音方向的角度 (`microphone_doa`)。常见的多麦克风数组主要有两类, 分别是均匀线性数组以及均匀圆形数组。以下皆为二维示意图(无高度差之影响)。

3.1.1. 均匀线性数组及坐标系

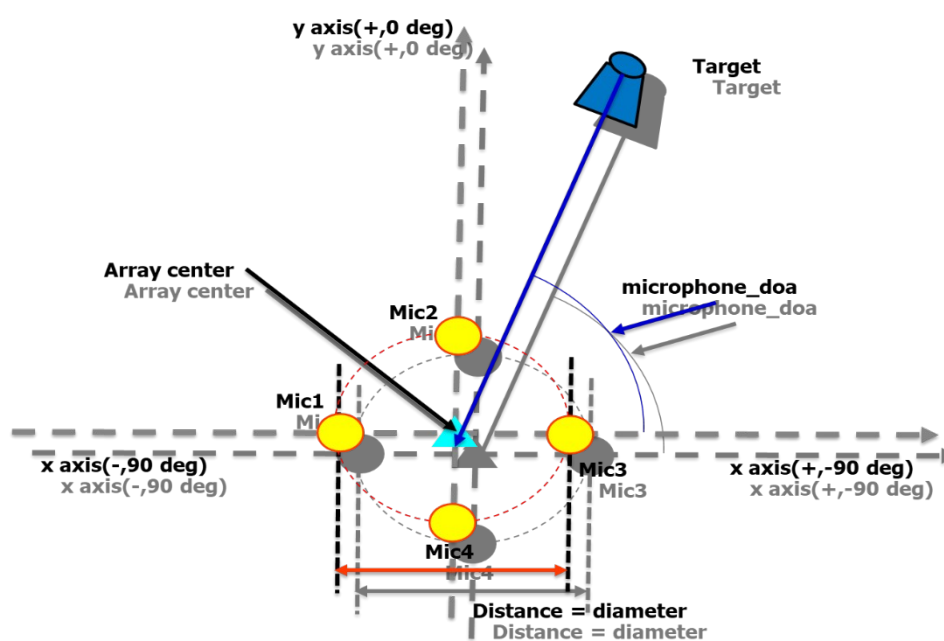
均匀线性数组为一直线排列, 且相邻麦克风之间距 `distance` 皆相同, 并且由于其具有对称性, 加强角度仅考虑上半平面 (-90 度至 90 度)。图二为均匀线性数组于空间坐标系之示意图。声音方向的角度 (`microphone_doa`) 定义为与数组中心及 x 轴之夹角。逆时针方向为正。`distance` 单位为公分, 建议至少大于 5 cm 或 6 cm。



图二 均匀线性数组及其坐标系

3.1.2. 均匀圆形数组及坐标系

均匀圆形数组为一圆形排列，且相邻麦克风与数组中心之夹角皆相同，麦克风间距即为圆形之直径，由于不具对称性，加强角度可以为整个平面，图三为均匀圆形数组于空间坐标系之示意图。声音方向的角度 (**microphone_doa**) 定义为与数组中心及 **x** 轴之夹角。逆时针方向为正。**distance** 为圆形的直径。单位为公分。建议至少大于 6 cm。



图三 均匀圆形数组及其坐标系

4. API 参考

4.1. 功能模块 API

API 名	功能
IaaBf_GetBufferSize	获取 Bf 算法运行需要的内存大小
IaaBf_Init	初始化 Bf 算法
IaaBf_SetConfig	配置 Bf 算法
IaaBf_GetConfig	获取 Bf 算法当前的配置参数信息
IaaBf_SetShape	Bf 指派麦克风数组形状

API 名	功能
IaaBf_Run	Bf 算法处理
IaaBf_Reset	重新初始化 Bf 算法
IaaBf_Free	释放 Bf 算法资源

4.2. IaaBf_GetBufferSize

➤ 功能

获取 Bf 算法运行所需要的内存大小。

➤ 语法

```
unsigned int IaaBf_GetBufferSize(void);
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

➤ 返回值

返回值为 Bf 算法运行所需要的内存大小

➤ 依赖

- 头文件: AudioBfProcess.h
- 库文件: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/ libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ 注意

该接口仅返回需要的内存大小，申请和释放内存的动作需应用来处理。

➤ 举例

请参考 [IaaBf_Run](#) 举例部分。

4.3. IaaBf_Init

➤ 功能

初始化 Bf 算法。

➤ 语法

```
BF_HANDLE IaaBf_Init(char* working_buffer,AudioBfInit* bf_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer	Bf 算法所使用的内存地址	输入
bf_init	Bf 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioBfProcess.h
- 库文件: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/ libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaBf_Run](#) 举例部分。

4.4. IaaBf_SetConfig

➤ 功能

配置 Bf 算法。

➤ 语法

```
ALGO_BF_RET IaaBf_SetConfig(BF_HANDLE handle,AudioBfConfig* bf_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Bf 算法 handle	输入
bf_config	Bf 算法的配置结构体指针	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件：AudioBfProcess.h
- 库文件：libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/ libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaBf_Run](#) 举例部分。

4.5. IaaBf_GetConfig

➤ 功能

获取 Bf 算法当前的配置参数。

➤ 语法

```
ALGO_BF_RET IaaBf_GetConfig(BF_HANDLE handle,AudioBfConfig* bf_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Bf 算法 handle	输入
bf_config	Bf 算法的配置结构体指针	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件：AudioBfProcess.h
- 库文件：libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/ libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaBf_Run](#) 举例部分。

4.6. IaaBf_SetShape

➤ 功能

Bf 指派麦克风数组形状。

➤ 语法

```
ALGO_BF_RET IaaBf_Setshape(BF_HANDLE handle,int shape);
```

➤ 形参

参数名称	描述	输入/输出
handle	Bf 算法 handle	输入
shape	指派数组形状。0:均匀线性数组，1:均匀圆形数组	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件：AudioBfProcess.h
- 库文件：libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/ libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ 注意

- 仅提供均匀线性数组及均匀圆型数组(示意图请看图一及图二)，客户有特殊数组需求，需告知修改
- 麦克风个数为 2 个时，只存在线性数组，其间距即为所设定麦克风间距
- 数组位置设定会强烈影响 **Beamforming** 效果，因此设定之数组位置必须与真实数组位置相同

➤ 举例

请参考 [IaaBf_Run](#) 举例部分。

4.7. IaaBf_Run

➤ 功能

BF 算法处理。

➤ 语法

```
ALGO_BF_RET IaaBf_Run(BF_HANDLE handle, short* microphone_input, short* microphone_output, float* microphone_doa);
```

➤ 形参

参数名称	描述	输入/输出
handle	Bf 算法 handle	输入
microphone_input	待进行声源定位的数据指针	输入
microphone_output	经过算法后的数据输出指针	输出
microphone_doa	Beamforming 处理的角度。此参数可由用户设定(较为直观)，也可由 SSL 计算(SSL 会计算接收讯号的时间差，还需要一个转换才能换至角度)。如果声音来自正前方，可输入 0。如果声音来自左右两侧，可输入 90 或-90	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件: AudioBfProcess.h

- 库文件: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/ libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ 注意

- 对于两颗麦克风数组(双声道音频可以负荷), microphone_input 指向的数据应以采样点为最小单位, 按照 L,R,L,R 的格式摆放, 数据长度必须和 IaaBf_Init 中设定的 point_number(即多少个采样点进行一次 beamforming 处理)相对应。→ 范例一
- 对于多颗麦克风数组(麦克风数多于两颗, 双声道音频无法负荷)之情况, 各个麦克风音频须为单声道, microphone_input 指向的数据仍应以采样点为最小单位, 并按照麦克风于数组的相对位置, 由左至右的格式摆放, 例如 4 颗线性麦克风数组, 摆放的方式为 [左一→左二→右二→右一], 数据长度必须和 IaaBf_Init 中设定的 point_number(即多少个采样点进行一次 beamforming 处理)相对应。→ 范例二
- microphone_output 数据长度必须和 IaaBf_Init 中设定的 Point_number 相对应。
 - ◆ 若 Point_number 为 128 点, 则 4 颗数组的 microphone_input 长度为 $128 \times 4 = 512$ 点, 输出 microphone_output 为 128 点。

➤ 举例

- 范例一: 两颗麦克风数组 (麦克风输入为双声道音频)

```

1. #include <stdio.h>
2. #include <unistd.h>
3. #include <fcntl.h>
4. #include <string.h>
5. #include <sys/time.h>
6. #include <sys/ioctl.h>
7. #include <stdlib.h>
8. #include "AudioBfProcess.h"
9.
10. #define USE_MALLOC (1)
11. typedef unsigned char      uint8;
12. typedef unsigned short    uint16;
13. typedef unsigned long     uint32;
14.
15. float AVERAGE_RUN(int a)
16. {
17.     static unsigned int num = 0;
18.     static float avg = 0;
19.     if(num == 0) avg = 0;
20.     num++;
21.     avg = avg + ((float)a - avg) / ((float)num);
22.     return avg;
23. }
24. unsigned int _OsCounterGetMs(void)
25. {
26.     struct timeval t1;
27.     gettimeofday(&t1, NULL);
28.     unsigned int T = ( (1000000 * t1.tv_sec) + t1.tv_usec ) / 1000;
29.     return T;

```

```
30. }
31.
32. int main(int argc, char *argv[])
33. {
34.     short input[256];
35.     short output[128];
36.     char inFile[512];
37.     char outFile[512];
38.
39.     FILE * fin, * fout;
40.     ALGO_BF_RET ret;
41.     int shape = 0;
42.     float direction = 0.0;
43.     int delay_sample = 0;
44.     float avg = 0;
45.     int counter = 0;
46.     unsigned int T0,T1;
47.     AudioBfInit bf_init;
48.     AudioBfConfig bf_config;
49.     BF_HANDLE handle;
50.     bf_init.mic_distance = 8.0;
51.     bf_init.point_number = 128;
52.     bf_init.sample_rate = 16000;
53.     bf_init.channel = 2;
54.     bf_config.noise_gate_dbfs = -20;
55.     bf_config.temperature = 20;
56.     bf_config.noise_estimation = 0;
57.     bf_config.output_gain = 0.7;
58.     bf_config.vad_enable = 0;
59.     bf_config.diagonal_loading = 10;
60.     #if USE_MALLOC
61.         char *WorkingBuffer2;
62.         WorkingBuffer2 = (char*)malloc(IaaBf_GetBufferSize());
63.     #endif
64.     handle = IaaBf_Init((char*)WorkingBuffer2, &bf_init);
65.     if (handle==NULL)
66.     {
67.         printf("BF init error\r\n");
68.         return -1;
69.     }
70.     else
71.     {
72.         printf("BF init succeed\r\n");
73.     }
74.     ret = IaaBf_SetConfig(handle,&bf_config);
75.     if (ret)
76.     {
77.         printf("Error occured in Config\r\n");
78.         return -1;
79.     }
80.     ret = IaaBf_SetShape(handle,shape);
81.     if (ret)
82.     {
83.         printf("Error occured in Array shape\r\n");
```

```

84.     return -1;
85. }
86. sprintf(infileName,"%s","../sample/data/Chn-14.wav");
87. sprintf(outfileName,"%s","./BFOut.pcm");
88.
89. fin = fopen(infileName, "rb");
90. if(!fin)
91. {   printf("the input file could not be open\n");
92.     return -1;
93. }
94.
95. fout = fopen(outfileName, "wb");
96. if(!fout)
97. {
98.     printf("the output file could not be open\n");
99.     return -1;
100. }
101.
102. fread(input, sizeof(char), 44, fin); // read header 44 bytes
103. fwrite(input, sizeof(char), 44, fout); // write 44 bytes output
104. // int delay_sample = 0;
105. while(fread(input, sizeof(short), bf_init.point_number*bf_init.channel, fin))
106. {
107.     counter++;
108.     T0 = (long)_OsCounterGetMs();
109.     ret = IaaBf_Run(handle,input,output,&direction);
110.     T1 = (long)_OsCounterGetMs();
111.     avg += (T1 - T0);
112.     if(ret)
113.     {
114.         printf("Error occured in Run\n");
115.         return -1;
116.     }
117.     fwrite(output, sizeof(short), bf_init.point_number, fout);
118. }
119. avg /= counter;
120. printf("AVG is %.2f ms\n",avg);
121. IaaBf_Free(handle);
122. fclose(fin);
123. fclose(fout);
124. free(WorkingBuffer2);
125. printf("Done\n");
126. return 0;
127. }

```

- 范例二：多颗麦克风数组(麦克风输入为4个单声道音讯)

```

1. #include <stdio.h>
2. #include <unistd.h>

```



```
3. #include <fcntl.h>
4. #include <string.h>
5. #include <sys/time.h>
6. #include <sys/ioctl.h>
7. #include <stdlib.h>
8. #include "AudioBfProcess.h"
9.
10. #define MIC_NUM (4)
11. #define USE_MALLOC (1)
12. typedef unsigned char      uint8;
13. typedef unsigned short    uint16;
14. typedef unsigned long     uint32;
15.
16. float AVERAGE_RUN(int a)
17. {
18.     static unsigned int num = 0;
19.     static float avg = 0;
20.     if(num == 0) avg = 0;
21.     num++;
22.     avg = avg + ((float)a - avg) / ((float)num);
23.     return avg;
24. }
25. unsigned int _OsCounterGetMs(void)
26. {
27.     struct timeval t1;
28.     gettimeofday(&t1, NULL);
29.     unsigned int T = ( (1000000 * t1.tv_sec) + t1.tv_usec ) / 1000;
30.     return T;
31. }
32.
33. int main(int argc, char *argv[])
34. {
35.     /*****Input file init*****/
36.     short input[512];
37.     short output[128];
38.     short input_tmp1[128], input_tmp2[128], input_tmp3[128], input_tmp4[128];
39.     FILE * fin0, * fin1, * fin2, * fin3, * fout;
40.
41.     char infileName[MIC_NUM][512];
42.     char outfileName[512];
43.     ALGO_BF_RET ret;
44.     int k;
45.     float avg = 0;
```

```
46. int counter = 0;
47. unsigned int T0, T1;
48. /*****common setting between SSL and BF*****/
49. int point_number = 128;
50. float microphone_distance = 4.0;
51. int temperature = 20;
52. int sample_rate = 16000;
53. int shape = 0;
54. // int delay_sample[MIC_NUM-1] = {0,0,0}; //channel-1
55. float direction = 0.0;
56. /*****BF data init*****/
57. #if USE_MALLOC
58. char *WorkingBuffer_BF;
59. WorkingBuffer_BF = (char*)malloc(IaaBf_GetBufferSize());
60. #endif
61.
62. AudioBfInit bf_init;
63. AudioBfConfig bf_config;
64. BF_HANDLE bf_handle;
65. bf_init.mic_distance = microphone_distance;
66. bf_init.point_number = point_number;
67. bf_init.sample_rate = sample_rate;
68. bf_init.channel = MIC_NUM;
69. bf_config.noise_gate_dbfs = -20;
70. bf_config.temperature = temperature;
71. bf_config.noise_estimation = 0;
72. bf_config.output_gain = 0.7;
73. bf_config.vad_enable = 0;
74. bf_config.diagonal_loading = 10;
75. bf_handle = IaaBf_Init((char*)WorkingBuffer_BF, &bf_init);
76. if (bf_handle==NULL)
77. {
78.     printf("BF init error\r\n");
79.     return -1;
80. }
81. else
82. {
83.     printf("BF init succeed\r\n");
84. }
85. ret = IaaBf_SetConfig(bf_handle,&bf_config);
86. if (ret)
87. {
88.     printf("Error occured in Config\n");
```

```
89.     return -1;
90. }
91. ret = IaaBf_SetShape(bf_handle,shape);
92. if (ret)
93. {
94.     printf("Error occured in Array shape\n");
95.     return -1;
96. }
97. /*****open input file and output file*****/
98. sprintf(infileName[0],"%s","../sample/data/Chn-01.wav");
99. sprintf(infileName[1],"%s","../sample/data/Chn-02.wav");
100.    sprintf(infileName[2],"%s","../sample/data/Chn-03.wav");
101.    sprintf(infileName[3],"%s","../sample/data/Chn-04.wav");
102.    sprintf(outfileName,"%s","../BFOut.pcm");
103.
104.    fin0 = fopen(infileName[0], "rb");
105.    if(!fin0)
106.    {
107.        printf("the input file 0 could not be open\n");
108.        return -1;
109.    }
110.
111.    fin1 = fopen(infileName[1], "rb");
112.    if(!fin1)
113.    {
114.        printf("the input file 1 could not be open\n");
115.        return -1;
116.    }
117.    fin2 = fopen(infileName[2], "rb");
118.    if(!fin2)
119.    {
120.        printf("the input file 2 could not be open\n");
121.        return -1;
122.    }
123.    fin3 = fopen(infileName[3], "rb");
124.    if(!fin3)
125.    {
126.        printf("the input file 3 could not be open\n");
127.        return -1;
128.    }
129.    fout = fopen(outfileName, "wb");
130.    if(!fout)
131.    {
```

```

132.     printf("the output file could not be open\n");
133.     return -1;
134. }
135. fread(input, sizeof(char), 44, fin0);
136. fread(input, sizeof(char), 44, fin1);
137. fread(input, sizeof(char), 44, fin2);
138. fread(input, sizeof(char), 44, fin3);
139. fwrite(input, sizeof(char), 44, fout);
140. short * input_ptr;
141.
142. while(fread(input_tmp1, sizeof(short), bf_init.point_number, fin0))
143. {
144.     fread(input_tmp2, sizeof(short), bf_init.point_number, fin1);
145.     fread(input_tmp3, sizeof(short), bf_init.point_number, fin2);
146.     fread(input_tmp4, sizeof(short), bf_init.point_number, fin3);
147.     input_ptr = input;
148.     for(k=0;k<point_number;k++)
149.     {
150.         *input_ptr = input_tmp1[k];
151.         input_ptr++;
152.         *input_ptr = input_tmp2[k];
153.         input_ptr++;
154.         *input_ptr = input_tmp3[k];
155.         input_ptr++;
156.         *input_ptr = input_tmp4[k];
157.         input_ptr++;
158.     }
159.     counter++;
160.     T0 = (long)_OsCounterGetMs();
161.     ret = IaaBf_Run(bf_handle, input, output, &direction);
162.     T1 = (long)_OsCounterGetMs();
163.     avg += (T1 - T0);
164.     if(ret)
165.     {
166.         printf("Error ocured in Run\n");
167.         return -1;
168.     }
169.     fwrite(output, sizeof(short), point_number, fout);
170. }
171. avg /= counter;
172. printf("AVG is %.2f ms\n", avg);
173. IaaBf_Free(bf_handle);
174. fclose(fin0);

```

```

175.    fclose(fin1);
176.    fclose(fin2);
177.    fclose(fin3);
178.    fclose(fout);
179.    free(WorkingBuffer_BF);
180.    printf("Done\n");
181.
182.    return 0;
183. }

```

4.8. IaaBf_Reset

➤ 功能

重新初始化 Bf 算法。

➤ 语法

```
BF_HANDLE IaaBf_Reset(BF_HANDLE working_buffer, AudioBfInit* bf_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer	Bf 算法运行的内存地址	输入
bf_init	Bf 算法的初始化参数结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioBfProcess.h
- 库文件: libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/ libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ 注意

无。

➤ 举例

无。

4.9. IaaBf_Free

➤ 功能

释放 Bf 算法资源。

➤ 语法

```
ALGO_BF_RET IaaBf_Free(BF_HANDLE handle);
```

➤ 形参

参数名称	描述	输入/输出
handle	Bf 算法 handle	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件：AudioBfProcess.h
- 库文件：libBF_2MIC_LINUX.so/ libBF_2MIC_LINUX.a/ libBF_4MIC_LINUX.so/libBF_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaBf_Run](#) 举例部分。

5. BF 数据类型

5.1. BF 模块相关数据类型定义

数据类型	定义
AudioBfInit	Bf 算法初始化参数结构体类型

AudioBfConfig	Bf 算法配置参数结构体类型
BF_HANDLE	Bf 算法句柄类型

5.2. AudioBfInit

➤ 说明

定义 Bf 算法的初始化参数类型。

➤ 定义

```
typedef struct
{
    unsigned int point_number;
    unsigned int sample_rate;
    float mic_distance;
    unsigned int channel;
}AudioBfInit;
```

➤ 成员

成员名称	描述
point_number	Bf 算法处理一次的采样点数
sample_rate	采样率，目前支持 8k/16k
mic_distance	相邻麦克风之间的间距，单位为 cm, 建议为 5cm 或 6cm
channel	通道数

※ 注意

无。

➤ 相关数据类型及接口

[IaaBf_Init](#)、[IaaBf_Reset](#)

5.3. AudioBfConfig

➤ 说明

定义 BF 算法的配置参数结构体类型。

➤ 定义

```
typedef struct
```

```

{
    unsigned int temperature;
    int noise_gate_dbfs;

    int noise_estimation;
    float output_gain;
    int vad_enable;
    int diagonal_loading;
}AudioBfConfig;

```

➤ 成员

成员名称	描述
temperature	环境温度（摄氏度） 摄氏度 = $(5/9) * (\text{华氏度} - 32)$ 步长为 1
noise_gate_dbfs	噪音阈值（dB） 低于此值将会把该帧作为噪声部分处理。 应小心处理此值。 此参数必须考虑实际麦克风所录制的音量大小。 若设置太高，容易造成语音失真。 若设定置太低，会影响 BF 的效果。 建议值为 -44，步长为 1。
noise_estimation	噪声的估计模式，输入值为 0 或 1。 0 为自适应模式 1 为平均模式 一般状况下推荐使用 0
output_gain	输出增益，范围[0,1] 设定为 0.7 为正常情况，设定为 1，会增加 4dB 建议值为 0.7，不增加任何增益
vad_enable	语音活检测(VAD)模式是否开启，输入值为 0 或 1。 若开启此设定，则利用 VAD 来计算每帧是否进入噪音估计处理。 若关闭此设定，则利用上述所设定的 noise_gate_dbfs 来决定是否视为噪音处理。
diagonal_loading	对角加载(Diagonal loading)，应小心处理此值设定 若设置太高，会影响算法效果。 若设置太低，容易造成语音失真。 建议值为 10。

※ 注意事项

- 无。

➤ 相关数据类型及接口

[IaaBf_SetConfig](#)、[IaaBf_GetConfig](#)

5.4. BF_HANDLE

➤ 说明

定义 Bf 算法的句柄类型。

➤ 定义

```
typedef void* BF_HANDLE;
```

➤ 成员

成员名称	描述
------	----

※ 注意事项

无

➤ 相关数据类型及接口

[IaaBf_Init](#)、[IaaBf_SetConfig](#)、[IaaBf_GetConfig](#)

[IaaBf_SetShape](#)、[IaaBf_Run](#)、[IaaBf_Reset](#)、[IaaBf_Free](#)

6. 错误码

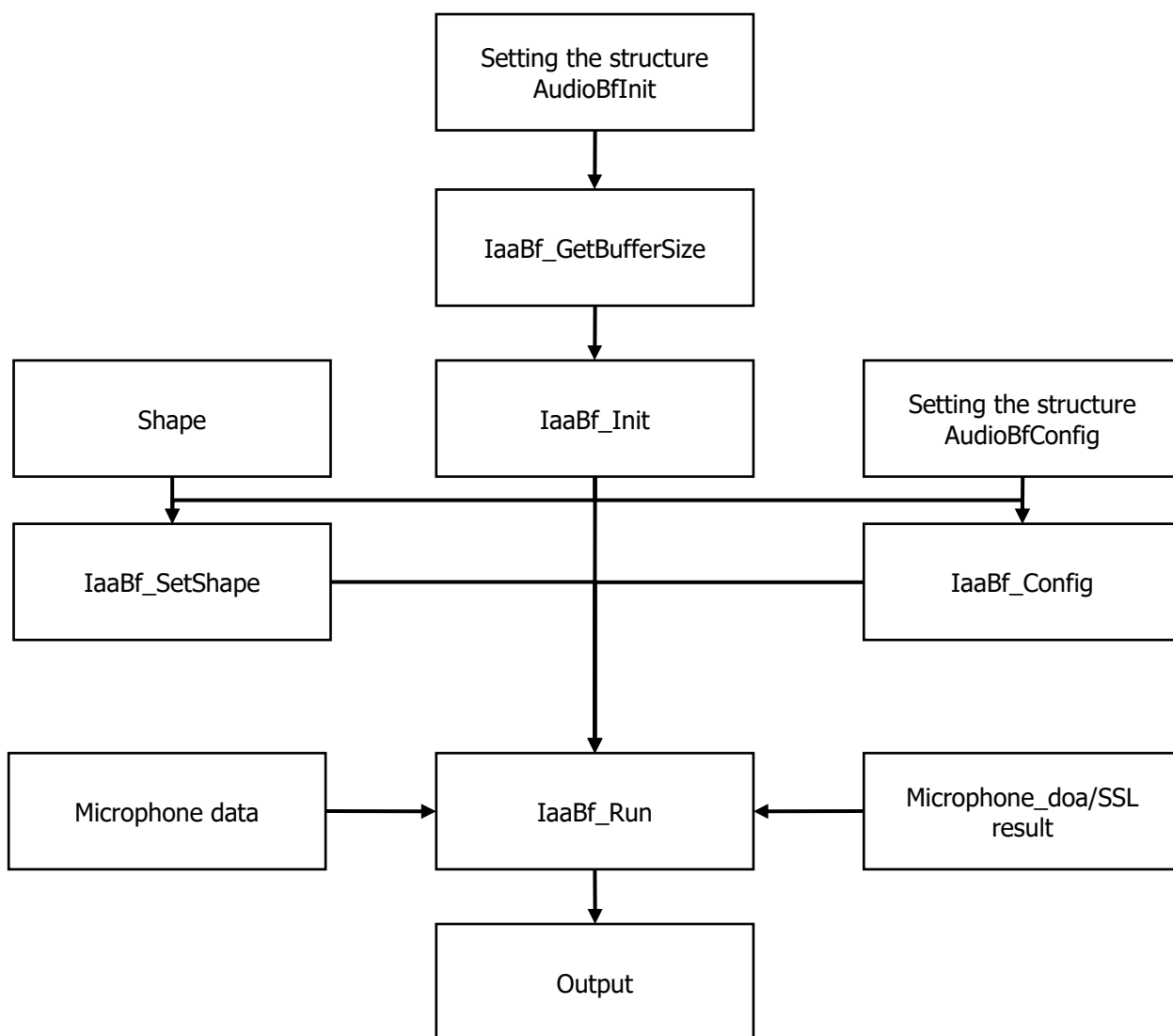
BF API 错误码如下表所示：

表 6-1 BF API 错误码

错误码	宏定义	描述
0x00000000	ALGO_BF_RET_SUCCESS	BF 执行成功
0x10000301	ALGO_BF_RET_INIT_ERROR	BF 初始化错误
0x10000302	ALGO_BF_RET_INVALID_CONFIG	BF Config 无效
0x10000303	ALGO_BF_RET_INVALID_HANDLE	BF Handle 无效
0x10000304	ALGO_BF_RET_INVALID_NOISE_ESTIMATION	BF 噪音估计模式设置无效

错误码	宏定义	描述
0x10000305	ALGO_BF_RET_INVALID_VAD_ENABLE	BF VAD 模式设置无效
0x10000306	ALGO_BF_RET_INVALID_OUTPUT_GAIN	BF 输出增益设置无效
0x10000307	ALGO_BF_RET_INVALID_INPUT_POINTER	BF 输入指标无效
0x10000308	ALGO_BF_RET_INVALID_SAMPLERATE	BF 采样率无效
0x10000309	ALGO_BF_RET_INVALID_POINTNUMBER	BF 采样点数无效
0x10000310	ALGO_BF_RET_INVALID_CHANNEL	BF 通道数无效
0x10000311	ALGO_BF_RET_INVALID_CALLING	BF 呼叫 API 顺序错误
0x10000312	ALGO_BF_RET_API_CONFLICT	有其他 API 正在运行
0x10000313	ALGO_BF_RET_INVALID_GEOMETRY_TYPE	BF 指派数组形状错误
0x10000314	ALGO_BF_RET_INVALID_MIC_DISTANCE	BF 指派麦克风间距错误

7. 流程图



图四 Beamforming 流程图